Introduction to version control with Git

Day 2: Branching, Merging and collaboration workflows

Selina Baldauf

November 10, 2026

Before we start

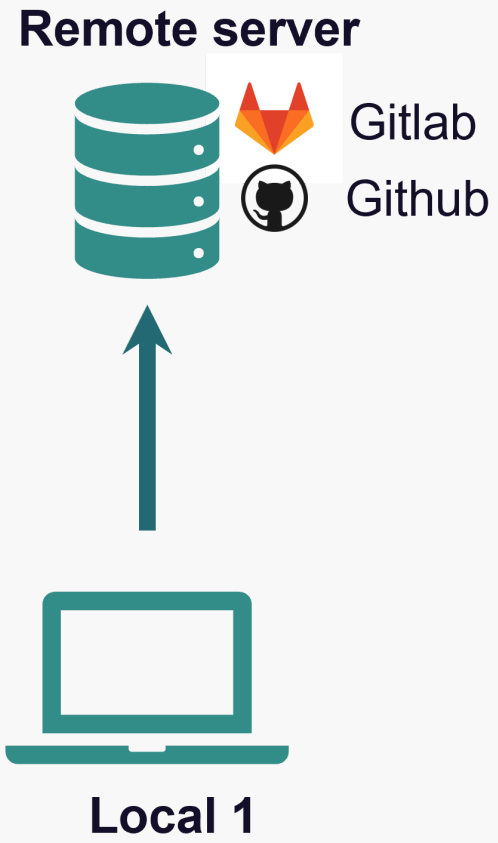
Let's check if we are all set with the teams.

Did you accept the invitation from your team mate? You need it to push to their repo.

Recap

Basic Git workflow:

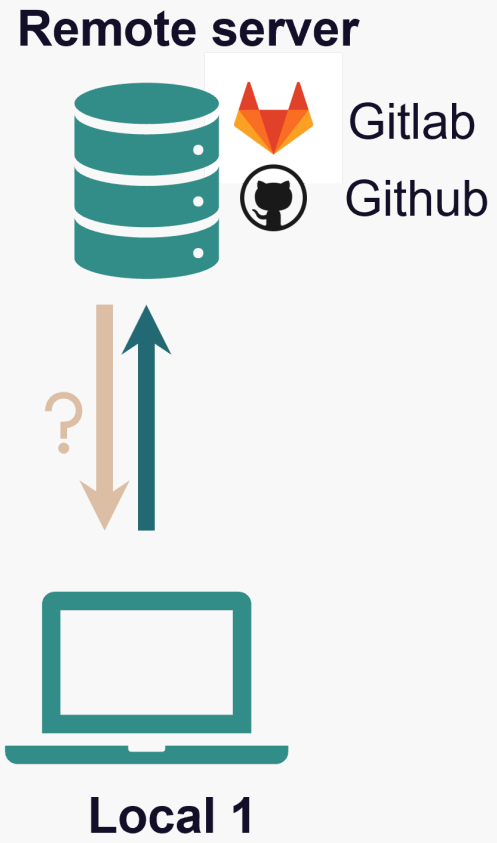
1. **Initialize** a Git repository
2. **Work** on the project
3. **Stage** and **commit** changes to the local repository
4. **Push** to the remote repository



Recap

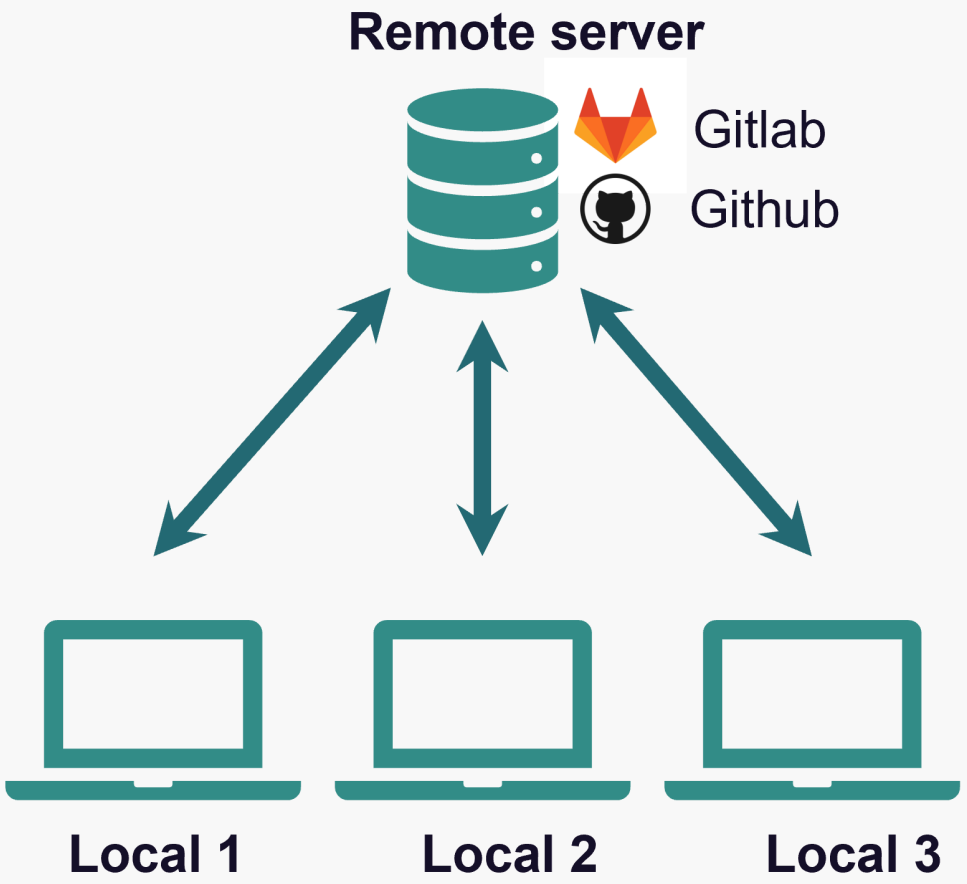
Basic Git workflow:

1. **Initialize** a Git repository
2. **Work** on the project
3. **Stage** and **commit** changes to the local repository
4. **Push** to the remote repository



Recap

Git is a **distributed version control system**

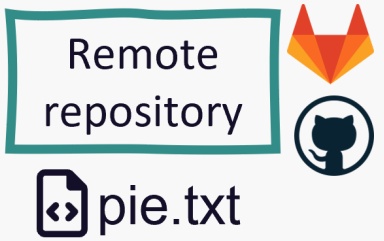


- Idea: many *local* repositories synced via one *remote* repo
- Collaborate with
  - **yourself** on different machines
  - your **colleagues** and friends
  - **strangers** on open source projects

Get a repo from a remote

In Git language, this is called **cloning**

- Get a **full copy** of the remote repo



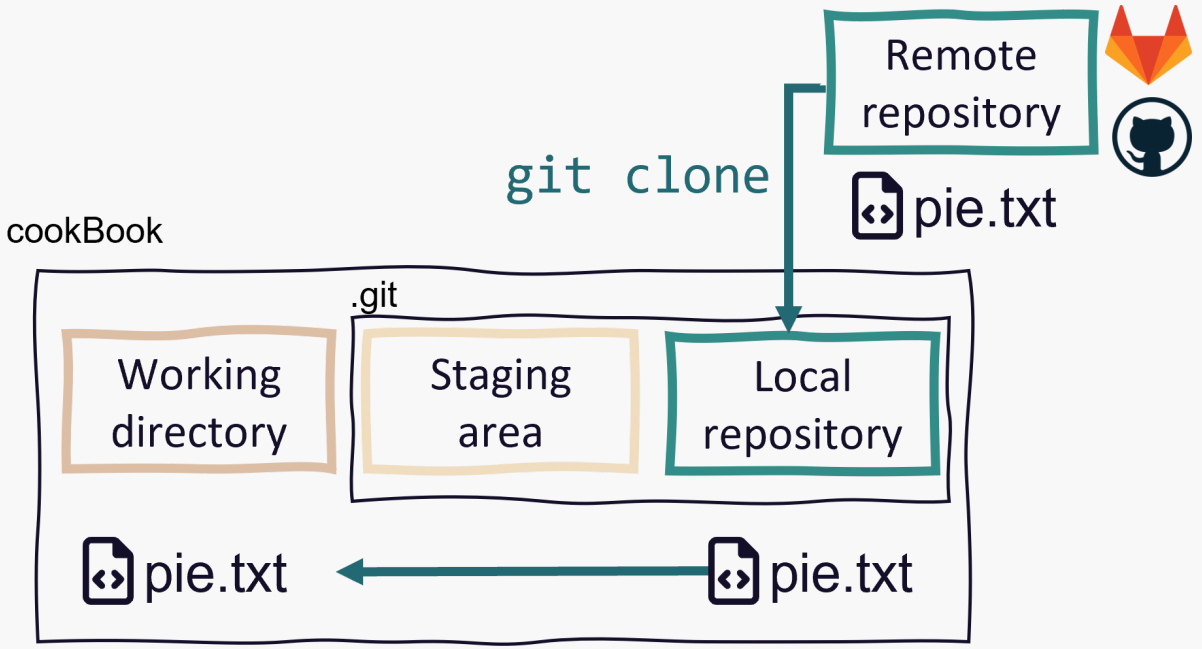




Get a repo from a remote

In Git language, this is called **cloning**

- Get a **full copy** of the remote repo



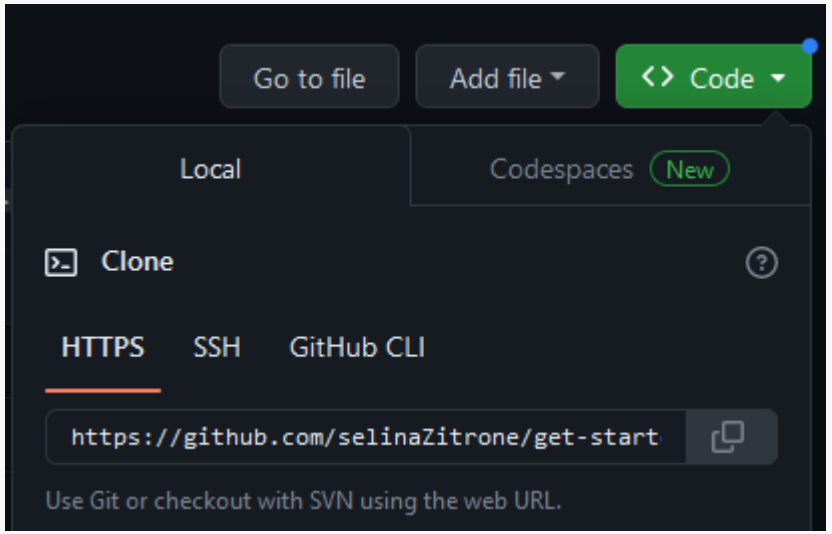


Get a repo from a remote

You can clone

- all of your own repositories (public and private)
- all repositories you are a collaborator on (public and private)
- all public repositories of other people
  - To contribute: **fork** the repo (your own copy on GitHub), then clone your fork

All you need is the URL of the remote repository

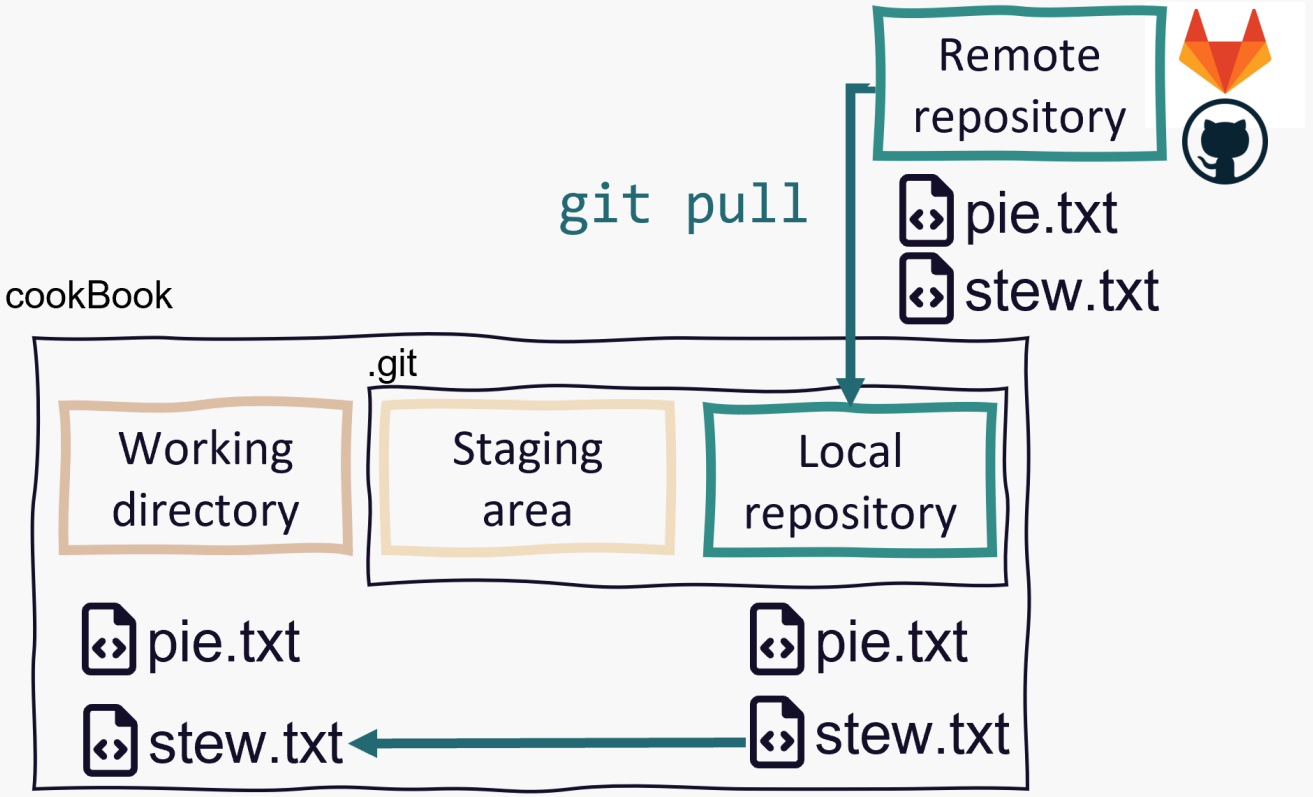

Go to file
Add file
Code
Local
Codespaces
New
Clone
HTTPS
SSH
GitHub CLI
https://github.com/selinaZitrone/get-start
Use Git or checkout with SVN using the web URL.

Now you (5 min)

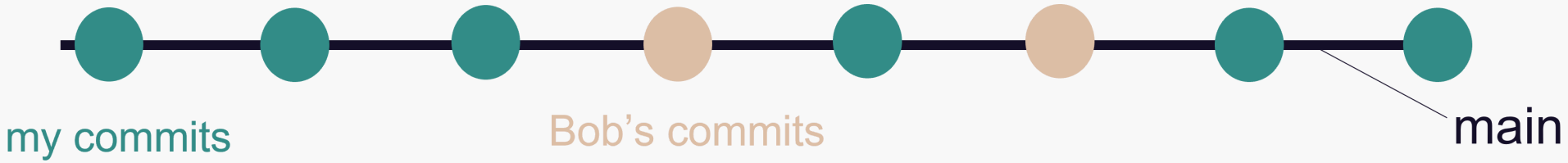
> Clone your team mate's cook book repo
> Details in Task 1 "Clone"

Get changes from the remote

- Local changes, publish to remote: `git push`
- Remote changes, pull to local: `git pull`

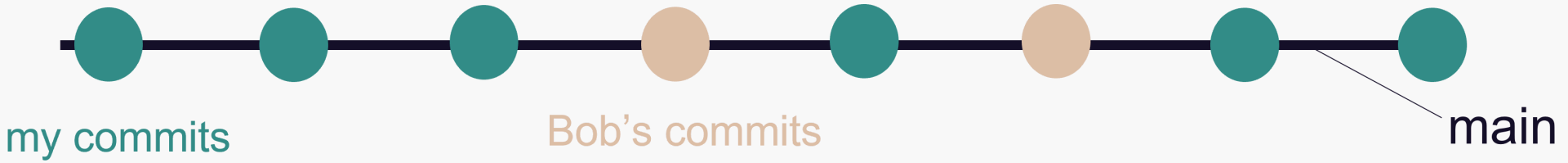


A simple collaboration workflow



- One remote repo on GitHub, multiple local repos (Bob and me)
- A **branch** is a separate line of commits. `main` is the default one.
- Idea: Everyone works on the same branch
  - **Pull before** you start **working**
  - **Push after** you finished **working**

A simple collaboration workflow



This works well if

- Repo is not updated often
- You don't work on the same files simultaneously
- No need to discuss changes before they are integrated
- You collaborate with yourself

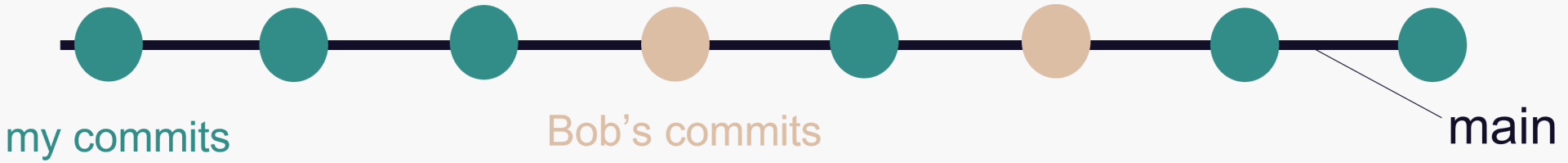
Let's give it a try

- Make sure you are in the repository of your team mate
- Open a recipe in the cook book of your team mate
  - **Repository -> Show in Explorer/Finder**
- Change something in there
- Commit the change and push it

Get the changes of your team mate from the remote.

- Switch to your own cook book repository
- Pull the changes (same place as the push button: **Pull origin**; if it says **Fetch origin**, click that first)
- Have a look at the commit history to see what changed

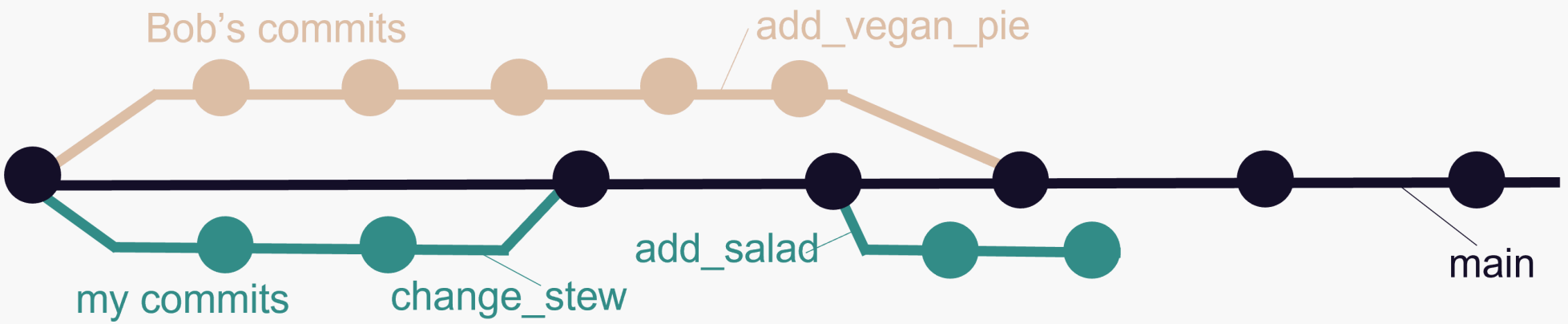
A simple collaboration workflow



This workflow starts to be problematic when

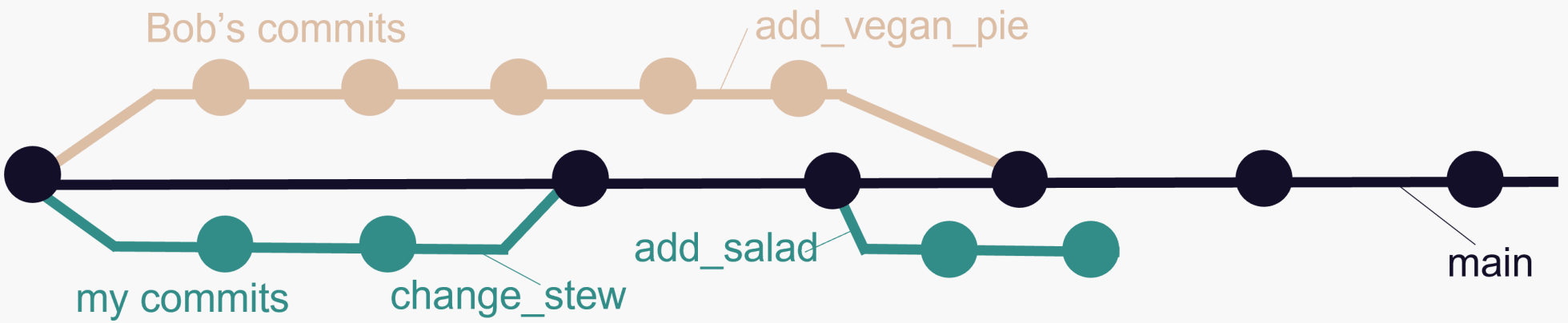
- People push often/forget to pull regularly
  - Potential conflicts on main (two people changed the same lines, see appendix)
- You just want to experiment
  - Everything goes directly to main

A branching-merging workflow



- One remote repo on GitHub, multiple local repos
- Idea: Everyone works on their **separate branch**
  - **Merge** branch with the main when work is done
- **Pull before** and **push after** working

A branching-merging workflow



Advantages of this approach

- Helps keep main working
- Potential conflicts don't have to be solved on main
- You can experiment without messing up the main

Working on a separate branch

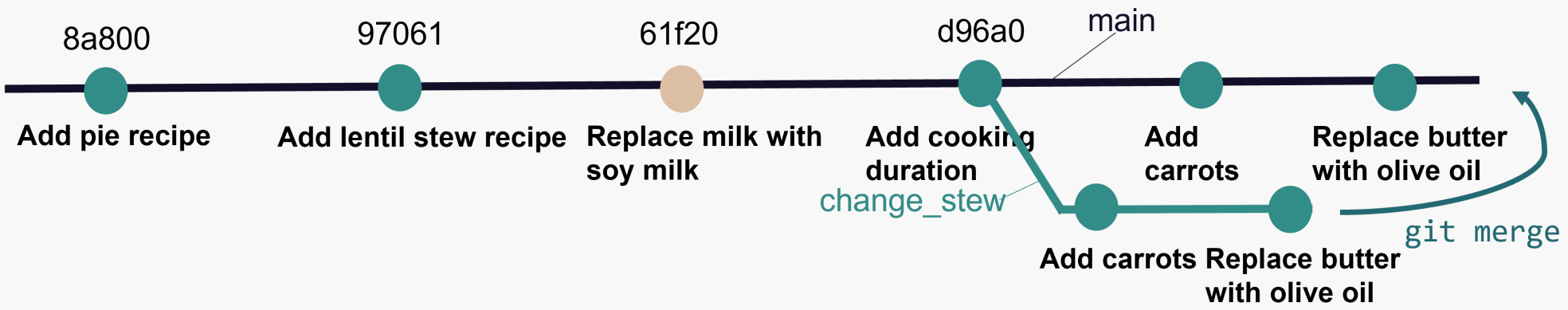
To create and work on a separate branch:



- Create a local branch and switch to it
- Work on the branch like you are used to
  - Make changes, **stage** and **commit**

Merging changes from a branch

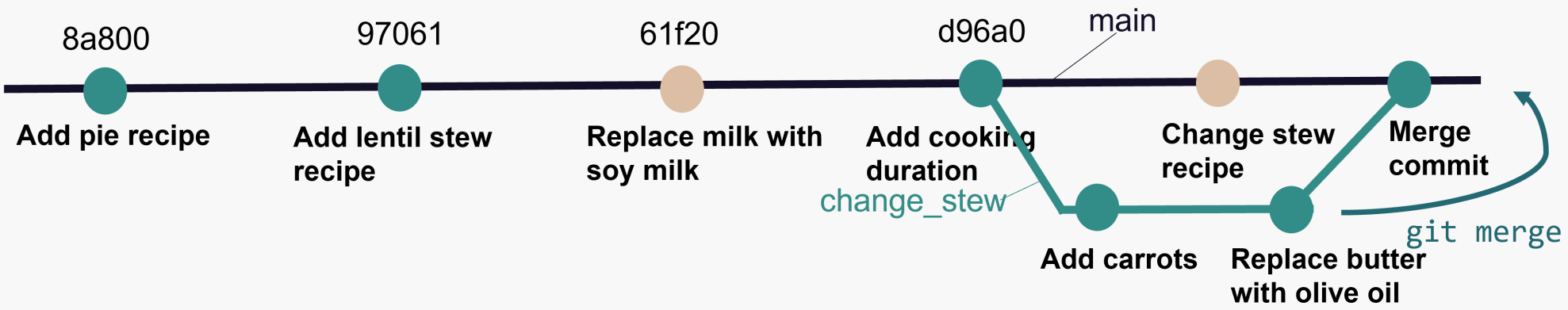
To bring changes to the main branch you need to **merge** them.



Git merge brings the commits from the branch to main

Merging changes from a branch

To bring changes to the main branch you need to **merge** them.

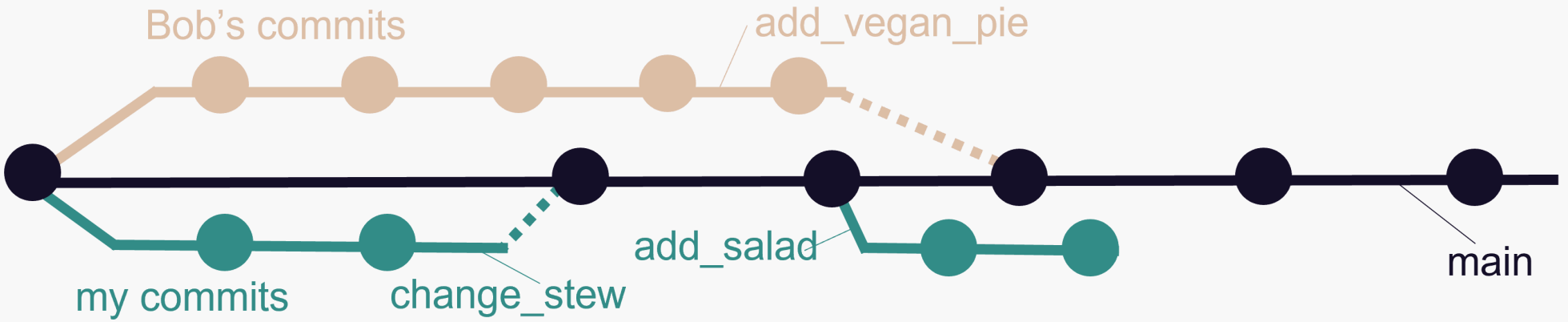


If there was a commit in main, a *merge commit* is introduced.

Now you (10 min)

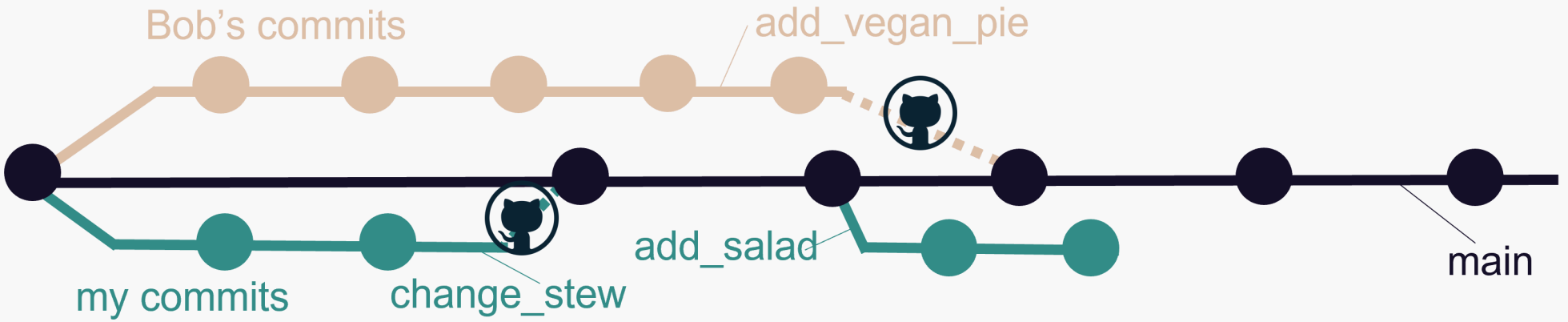
> Create a branch and merge it in your team mate's cook book
> Complete task 2 "Branch and merge"

A branching-merging workflow with GitHub



- One remote repo on GitHub, multiple local repos
- Idea: Everyone works on their separate branch
  - ~~**Merge** branch with the main when work is done~~

A branching-merging workflow with GitHub



- One remote repo on GitHub, multiple local repos
- Idea: Everyone works on their separate branch
  - ~~**Merge** branch with the main when work is done~~
  - **Create a pull request** on GitHub to ask for a merge
- **Pull before** and **push after** working

A branching-merging workflow with GitHub

A pull request is basically asking your collaborators:

> What do you think of my changes? Can we integrate them in main or do we still need to change something?

GitHub has nice features for pull requests, e.g.:

- **Provide context and explanations** for your changes
- Collaborators can easily **compare versions**
- Collaborators can **discuss and comment** on your changes

A pull request is merged on GitHub when **everyone agreed on the code**.

Now you (10 min)

> Create a pull request on your team mate's repo
>
> Complete task 3 "Pull requests"

Now you (5 min)

Review and merge the pull request in your own repo

Complete task 4 “Review and merge”

Some good practice tips

Git

- Commit often (small changes that can be described in one commit message)
- Write good commit messages (it becomes a habit)
- Push (at least) daily (backup!)
- Use `.gitignore`
- Don’t commit secrets ;)

Publishing

Some essentials that will improve your published repository:

- Expand your README.md file
  - Tell people what your project is about and how to use it
- Add a LICENSE file
  - Tell others how to use your code
- Add a DOI to your repository (e.g. via Zenodo)

If you are interested, browse some nice GitHub repositories for inspiration (e.g. Computational notebooks guide)

What now?

- Git can seem complicated at first, but you will get used to it
- Make it a habit to use Git, start with your own projects
- Improve your workflow step by step
- Practice makes perfect

Check out the resources page and the different How-tos for more info and practice.

Thanks for your attention

> Questions?

Appendix: Merge conflicts

Merge conflicts



Artwork by @allison_horst

When do conflicts happen?

- Two changes to the **same lines of the same file**
  - e.g. on two branches that you want to merge
  - or in your local and the remote repo when you pull
- Git cannot decide which version to keep, so it **asks you**
- Nothing is lost and you can always abort the merge

What a conflict looks like

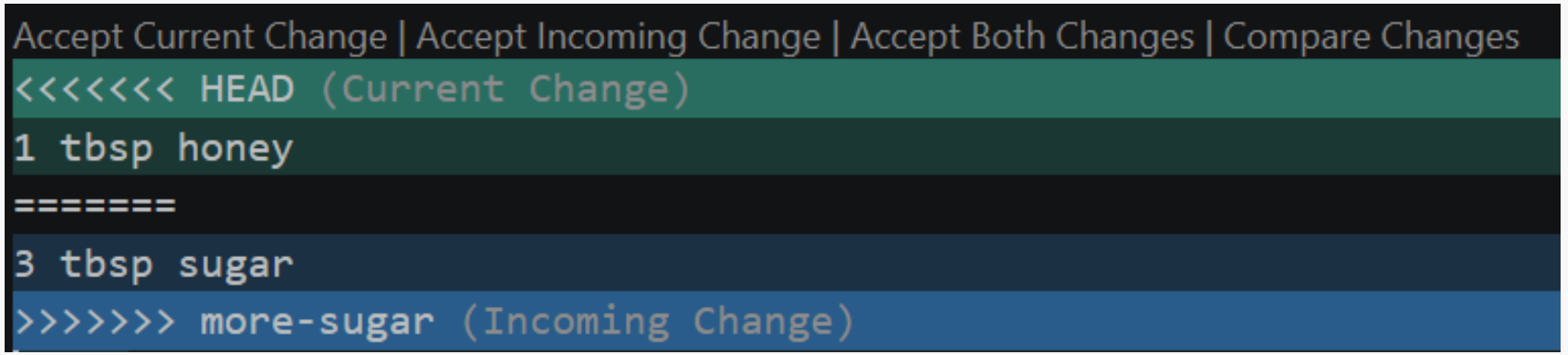
Git marks the conflicting lines in the file:

```
1 tsp vanilla extract
<<<<<<< HEAD
1 tbsp honey
=======
3 tbsp sugar
>>>>>>> more-sugar
Pinch of salt
```

- Between `<<<<<<< HEAD` and `=======`: version on your current branch (main)
- Between `=======` and `>>>>>>> more-sugar`: version on the branch you merge

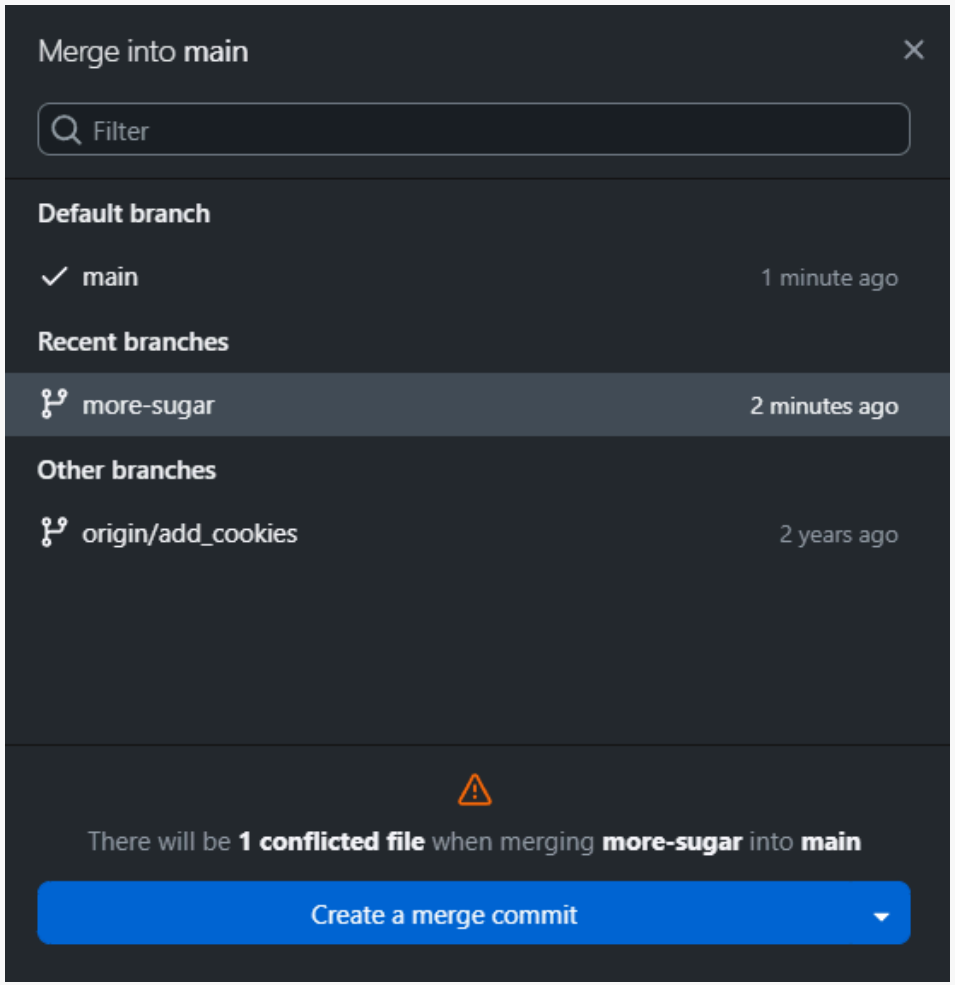
Editors can help

Many editors highlight conflicts and let you choose a version with one click, e.g. VS Code:

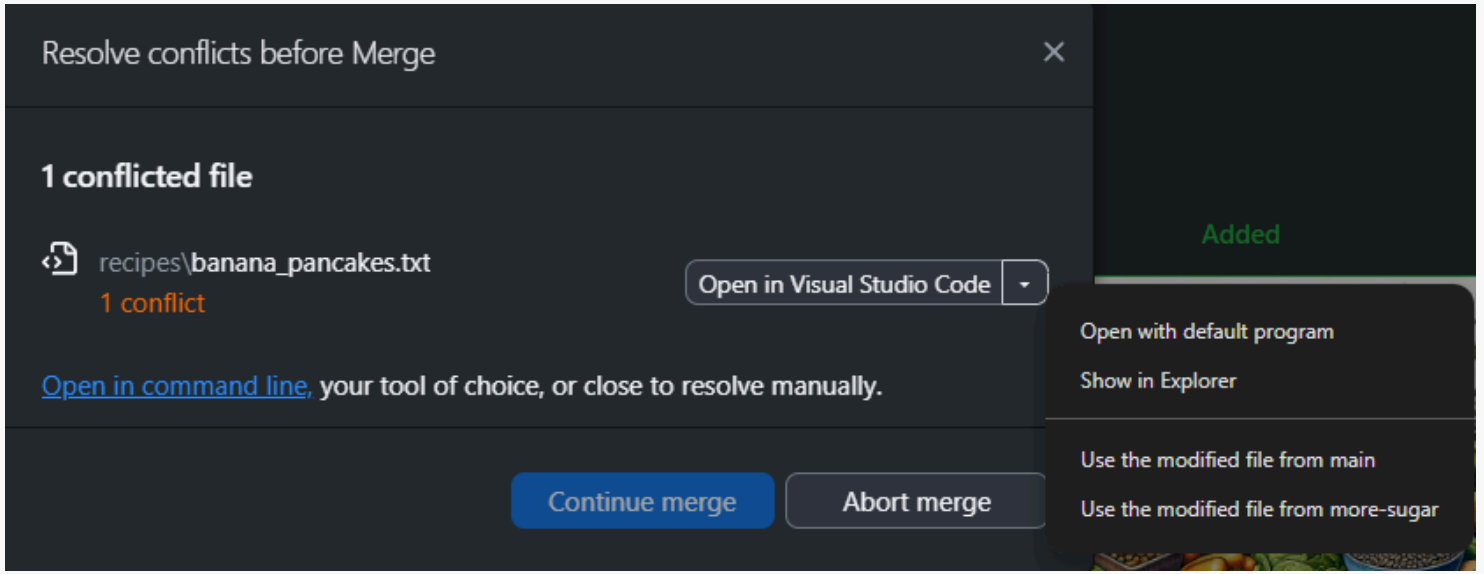


Resolve conflicts in GitHub Desktop

GitHub Desktop warns you before the merge



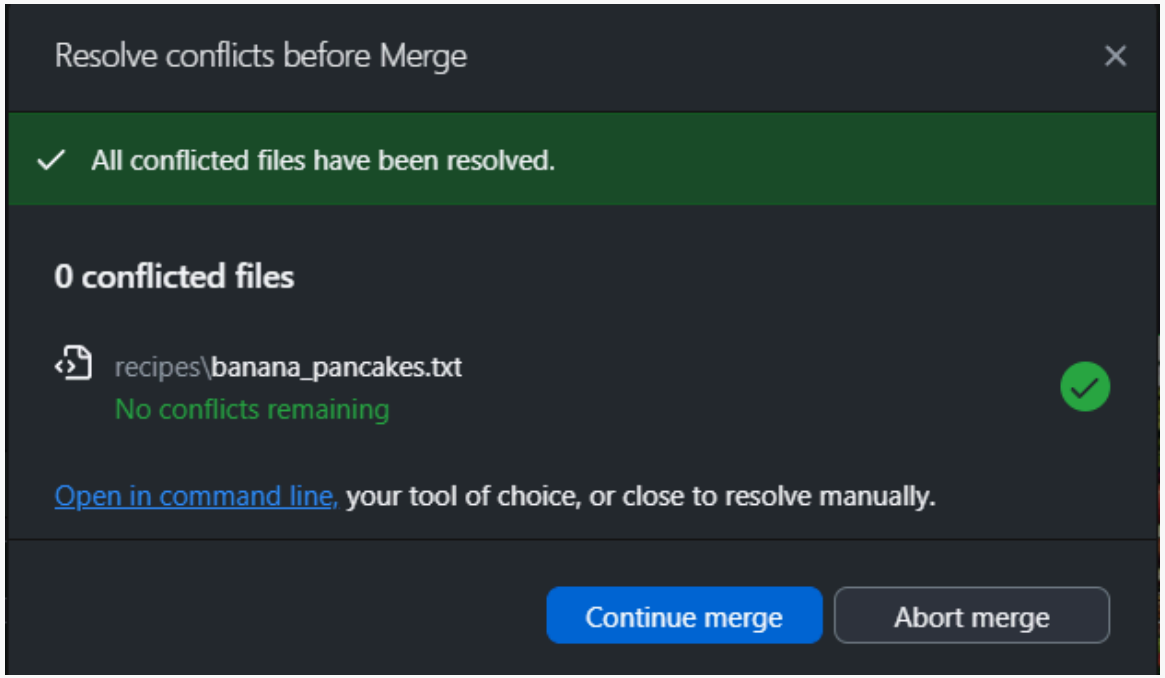
Resolve conflicts in GitHub Desktop



1. Open the file in your editor, keep what you want (or combine both) and **delete the markers**
2. Or use the dropdown to keep the whole file from one branch

Resolve conflicts in GitHub Desktop

When all conflicts are resolved, finish the merge with **Continue merge** (or go back with **Abort merge**)



Resolve conflicts in the terminal

- `git merge more-sugar` reports the conflict
- `git status` shows the conflicted files
- Open the file, keep what you want and delete the markers
- `git add <file>` and `git commit` to finish the merge
- Or go back with `git merge --abort`

How to avoid conflicts

- **Pull before** you start working
- **Commit and push often** (small changes)
- Keep branches **short-lived** and merge them regularly
- **Talk to your collaborators**: Who works on which file?